

RoboMaster Assessment 1

Networking, SDK Teleoperation, ROS 2 Integration,
Odometry Analysis and Mecanum Kinematics

Student	Faisal Zahid
Platform	DJI RoboMaster EP
Software	Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic, RoboMaster SDK
Evidence date	24 September 2026
Report status	Final submission

Assessment outcome

A complete end-to-end workflow was demonstrated: direct SDK control and telemetry logging, RNDIS-versus-AP network characterization, ROS 2 driver integration, keyboard teleoperation, rosbag recording, odometry analysis, and TF verification.

Executive Summary

This report documents the implementation and validation of a RoboMaster EP control and measurement workflow. The robot was first operated directly through the Python SDK using a purpose-built keyboard controller that logs battery, attitude, and four wheel speeds at 10 Hz. The same 30 s experiment was repeated over USB RNDIS and Wi-Fi access-point (AP) links. The ROS 2 Jazzy driver was then built and used to command the robot, record odometry, command velocity, and joint states, and analyze motion tracking and the transform tree.

Evidence item	Measured result	Interpretation
RNDIS ping	0.442 ms mean; 0% loss	Very low-latency local link
AP ping	14.069 ms mean; 0% loss	Reliable but 31.78x higher latency
Telemetry cadence	99.881 ms RNDIS; 99.823 ms AP	Both met the 10 Hz target
ROS bag	129.484 s; 2,568 messages	All required topics captured
Odometry stream	1,260 messages; 9.723 Hz	Stable near-10 Hz feedback
Tracking discrepancy	0.058 m final; 13.44 deg yaw	Small positional offset; heading accumulated

Main finding

The wired RNDIS connection was decisively better for latency-sensitive control, while both links maintained almost identical telemetry sampling cadence. ROS 2 operation was successful after aligning the package entry-point interpreter with the virtual environment and installing the missing joint-state publisher package.

Evidence map

Requirement	Evidence used
Standalone control and safety	ep_teleop_sdk.py and 30 s CSV logs
Network comparison	ping_rndis.txt, ping_ap.txt, network_comparison.png
ROS 2 topics and motion	ros2_ap_run_complete_4 MCAP and metadata
Trajectory and drift	ros2_trajectory.png and ros2_tracking_drift.png
Frame structure	robomaster_tf_tree.pdf and Graphviz source

1. System Setup and Installation

1.1 Software environment

Component	Selected version / role
Operating system	Ubuntu 24.04 LTS (Noble)
ROS middleware	ROS 2 Jazzy Jalisco
Simulator	Gazebo Harmonic, the recommended pairing for Jazzy
Python isolation	~/rm_venv using Python 3.12
Robot driver	jeguzzi/robomaster_ros, built in ~/ros2_ws
Robot SDK	DJI RoboMaster Python SDK installed inside rm_venv

1.2 Installation sequence

```
sudo apt update
sudo apt install ros-jazzy-desktop ros-dev-tools
sudo apt install ros-jazzy-ros-gz ros-jazzy-joint-state-publisher

python3 -m venv ~/rm_venv --system-site-packages
source ~/rm_venv/bin/activate
python -m pip install --upgrade pip
python -m pip install robomaster pygame 'numpy<2'

mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
git clone https://github.com/jeguzzi/robomaster_ros.git
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
~/rm_venv/bin/python /usr/bin/colcon build --symlink-install
source install/setup.bash
```

1.3 NumPy ABI compatibility

Compiled Python extensions target a specific NumPy C application binary interface (ABI). A binary built against NumPy 1.x can fail when imported with NumPy 2.x because internal C structures and symbols changed. This is not a normal Python syntax error: the package may install successfully but fail only when its native extension is loaded. Pinning `numpy<2` keeps the runtime ABI compatible with older precompiled dependencies; rebuilding every affected extension against NumPy 2 is the longer-term alternative.

Reproducibility rule

Activate `rm_venv` first, then source `/opt/ros/jazzy/setup.bash` and the workspace overlay. The package entry-point shebang must resolve to `~/rm_venv/bin/python` so that the installed RoboMaster SDK is importable.

2. Standalone SDK Teleoperation

2.1 Control architecture

The controller separates asynchronous robot callbacks from deterministic CSV sampling. SDK callbacks update a lock-protected telemetry object. A dedicated logger thread snapshots the latest state every 0.1 s using a monotonic schedule, while the Pygame event loop runs at 50 Hz and continuously sends chassis velocity commands.

Input	Command	Safety behavior
W / S	+/- longitudinal speed x	Released key returns x to zero
A / D	+/- lateral speed y	Holonomic strafe; released key returns y to zero
Q / E	+/- yaw speed z	rad/s converted to deg/s for SDK
Space	Immediate zero command	Program continues for controlled recovery
Esc / window close	Exit	Zero command, unsubscribe, close robot, save CSV

2.2 Telemetry schema

Field group	CSV columns	Units
Timing	host_unix_time_s, elapsed_s	s
Power	battery_pct	%
Attitude	yaw_deg, pitch_deg, roll_deg	deg
Drive feedback	wheel_1_rpm ... wheel_4_rpm	rpm

2.3 Critical implementation details

- The logger advances an absolute monotonic deadline instead of repeatedly sleeping for 0.1 s, limiting cumulative timing drift.
- A lock makes each CSV row a coherent snapshot rather than a mixture of callback updates.
- Command timeout is 0.15 s; stale commands therefore decay quickly if the control loop stops.
- The finally block attempts a zero-speed command even after an exception or Ctrl-C.
- Speed validation limits translation to 1.0 m/s and yaw to 2.0 rad/s.

```
# Core mapping and SDK conversion
vx = linear_speed * (pressed[K_w] - pressed[K_s])
vy = linear_speed * (pressed[K_a] - pressed[K_d])
wz = yaw_speed * (pressed[K_q] - pressed[K_e])
chassis.drive_speed(x=vx, y=vy, z=degrees(wz), timeout=0.15)
```

3. RNDIS and Wi-Fi AP Network Experiment

3.1 Method

For each connection mode the same controller subscribed to battery, attitude, and ESC feedback at 10 Hz and recorded a 30 s CSV log. Separately, 100 ICMP echo requests were sent to the robot. RNDIS used the USB-network endpoint 192.168.42.2; AP mode used 192.168.2.1. Packet loss, round-trip time, and telemetry inter-arrival time were compared.

Metric	RNDIS	Wi-Fi AP	Result
Telemetry intervals	n = 300	n = 300	301 samples per CSV
Mean interval	99.8807 ms	99.8232 ms	Both approximately 10 Hz
Interval stdev	7.4185 ms	7.9583 ms	Similar application-level jitter
Ping packets	100 / 100	100 / 100	0% loss for both
Ping min / mean / max	0.289 / 0.442 / 1.633 ms	3.297 / 14.069 / 25.035 ms	RNDIS much faster
Ping mdev	0.185 ms	5.841 ms	AP is more variable
Mean latency ratio	1.00x	31.78x	AP / RNDIS

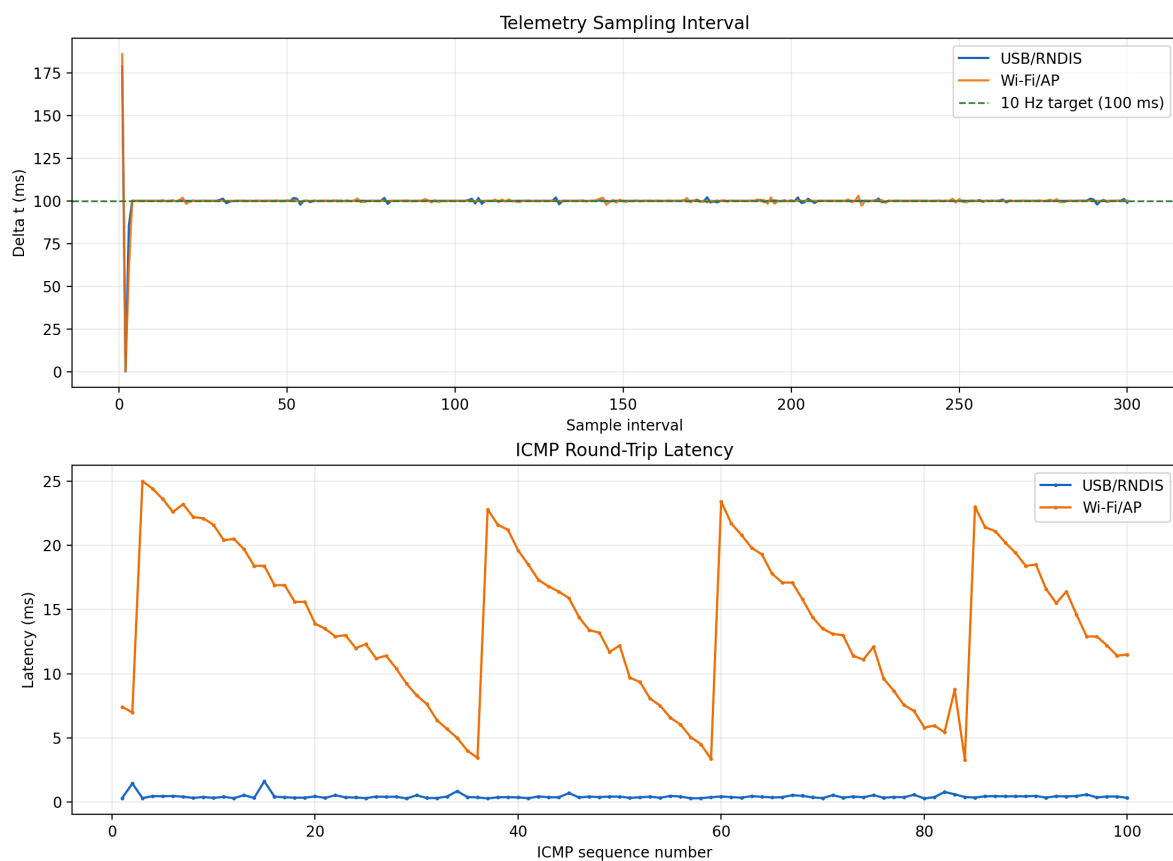


Figure 1. Measured telemetry cadence and ping latency for RNDIS and AP links.

3. Network Results and Interpretation

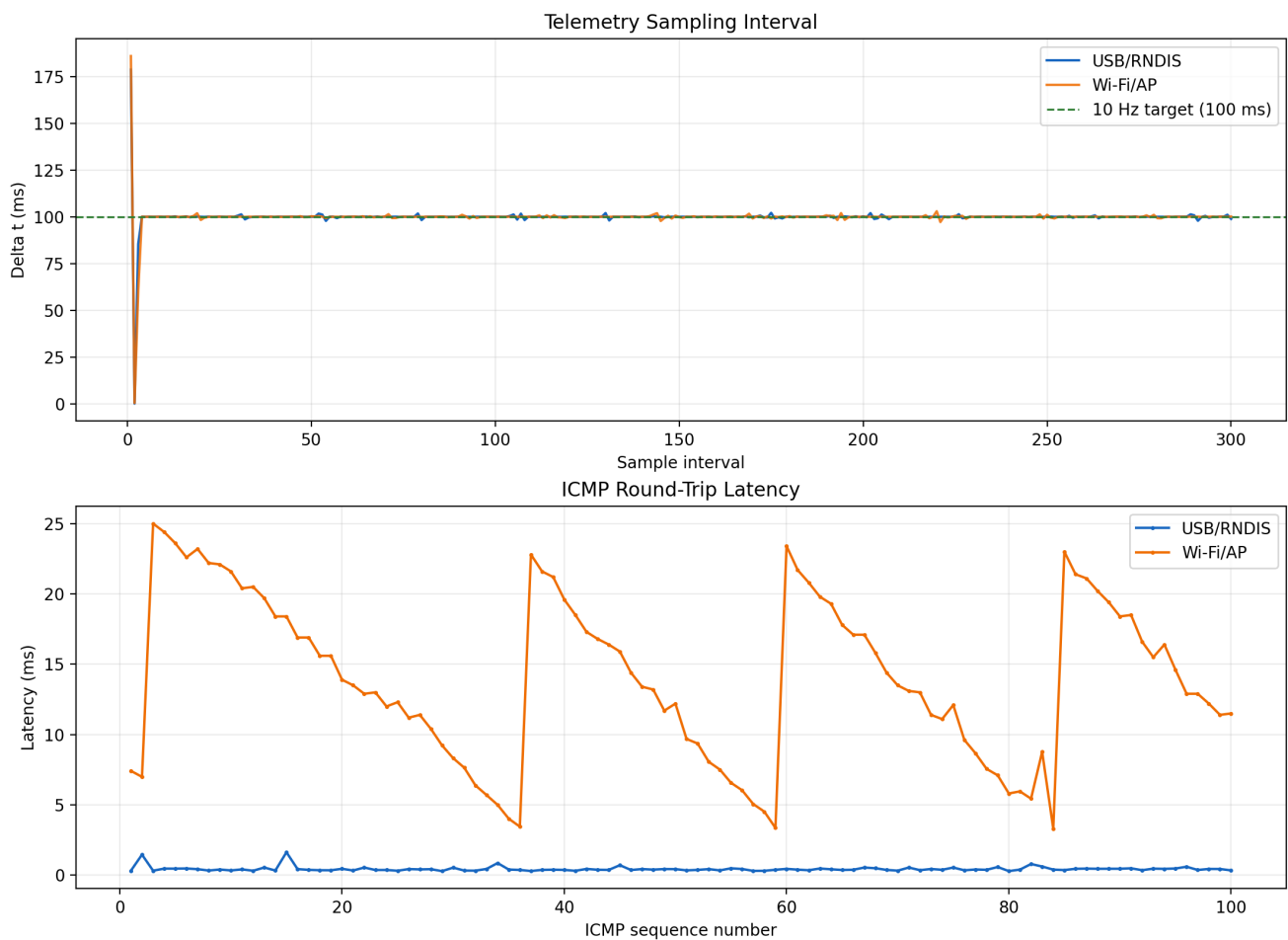


Figure 2. Full-size comparison. Telemetry cadence remains near 100 ms on both links, while network RTT separates clearly.

Interpretation

The telemetry scheduler is primarily governed by the 10 Hz SDK subscription and host logger, so its average interval remains almost unchanged across the two transports. Ping probes isolate transport responsiveness more directly: the wired RNDIS mean of 0.442 ms is approximately 31.78 times lower than the AP mean of 14.069 ms. Both links achieved zero packet loss in this controlled test, so the distinction is latency and jitter rather than reachability.

Engineering decision

Use RNDIS for latency-sensitive experiments, repeatable timing, and software debugging. Use AP mode when mobility and cable-free operation matter more than minimum latency. The measured 10 Hz telemetry stream was viable in either mode.

Threats to validity

- A 100-packet test is a short snapshot and does not characterize congestion over hours or across different rooms.
- ICMP round-trip latency is not identical to end-to-end command actuation latency.
- Wireless results depend on interference, channel occupancy, distance, antenna orientation, and host power management.
- The telemetry interval includes SDK and host scheduling behavior, not only link delay.

4. ROS 2 Driver Integration and Recording

4.1 Interpreter and build issue resolved

The first installed discover entry point used `#!/usr/bin/python3`. The system interpreter could not import the RoboMaster SDK installed in `~/rm_venv`, producing `ModuleNotFoundError: robomaster`. Building through the virtual-environment interpreter regenerated the entry point with the correct shebang. The launch also required `ros-jazzy-joint-state-publisher`, which was installed through `apt`.

```
source ~/rm_venv/bin/activate
source /opt/ros/jazzy/setup.bash
cd ~/ros2_ws
~/rm_venv/bin/python /usr/bin/colcon build \
  --symlink-install --packages-select robomaster_ros
source install/setup.bash

ros2 launch robomaster_ros main.launch \
  model:=ep conn_type:=ap video:=0 audio:=0
```

4.2 Operational evidence

The driver reported a successful connection and enabled the Arm, Battery, Camera, Chassis, Gripper, LED, and Speaker modules. The ROS graph exposed command, sensing, description, and transform topics including `/cmd_vel`, `/odom`, `/joint_states`, `/tf`, and `/tf_static`. Keyboard commands were published using `teleop_twist_keyboard`.

Recorded topic	Type	Count
<code>/cmd_vel</code>	<code>geometry_msgs/msg/Twist</code>	20
<code>/odom</code>	<code>nav_msgs/msg/Odometry</code>	1,260
<code>/joint_states</code>	<code>sensor_msgs/msg/JointState</code>	1,288

Bag integrity
`ros2_ap_run_complete_4` contains `metadata.yaml` and a 2.3 MiB MCAP file. Duration is 129.484 s with 2,568 messages; every required topic is present.

5. Motion and Odometry Analysis

5.1 Trajectory reconstruction

The measured path is taken from odometry pose samples. A command-reference path is generated by integrating planar Twist commands with a piecewise-constant assumption between command updates. Both paths are shifted to a common origin so that the comparison emphasizes relative motion rather than the arbitrary initial odometry frame.

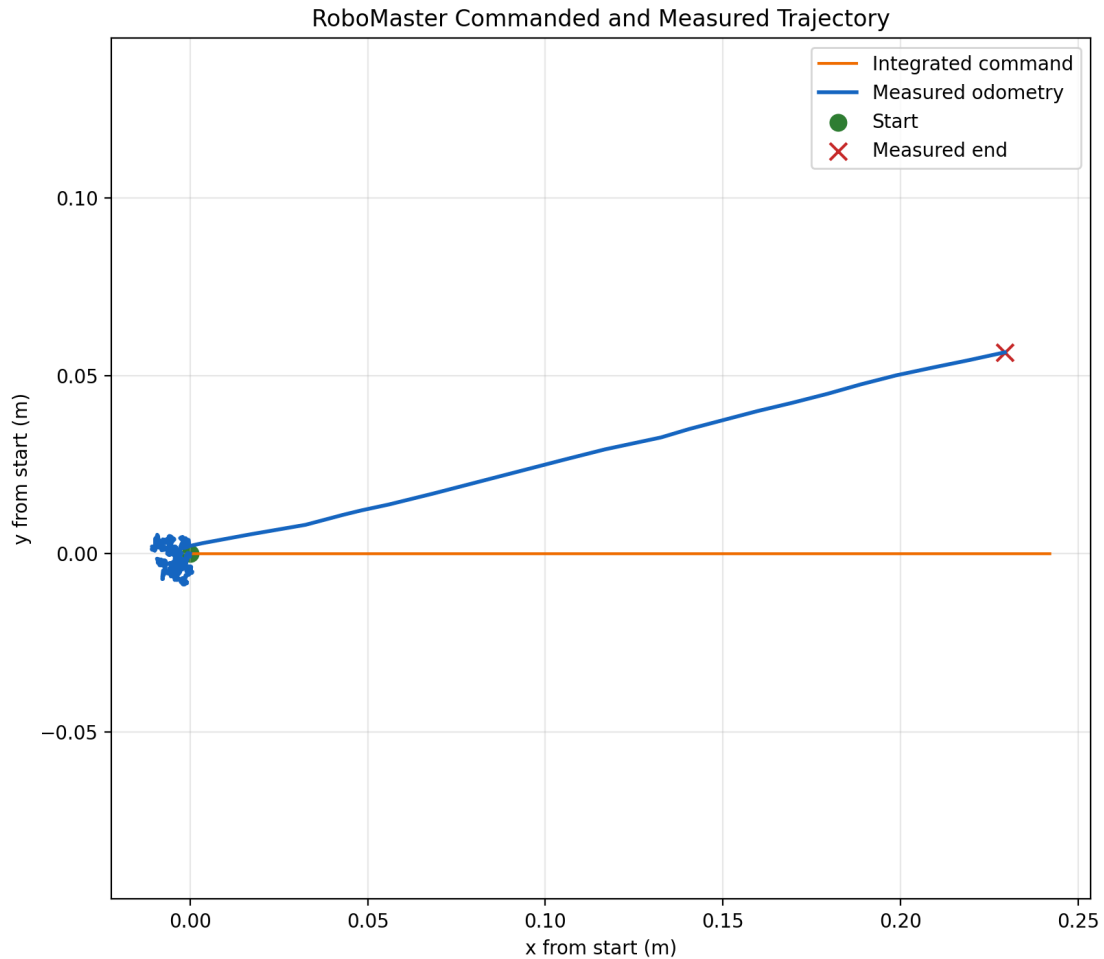


Figure 3. Command-integrated reference and measured odometry trajectory.

ROS metric	Value
Bag duration	129.484 s
Mean odometry interval	102.847 ms
Mean odometry rate	9.723 Hz
Final position discrepancy	0.0580 m
Maximum position discrepancy	0.0590 m
Final absolute yaw discrepancy	13.440 deg
Maximum absolute yaw discrepancy	13.510 deg

5. Tracking Discrepancy

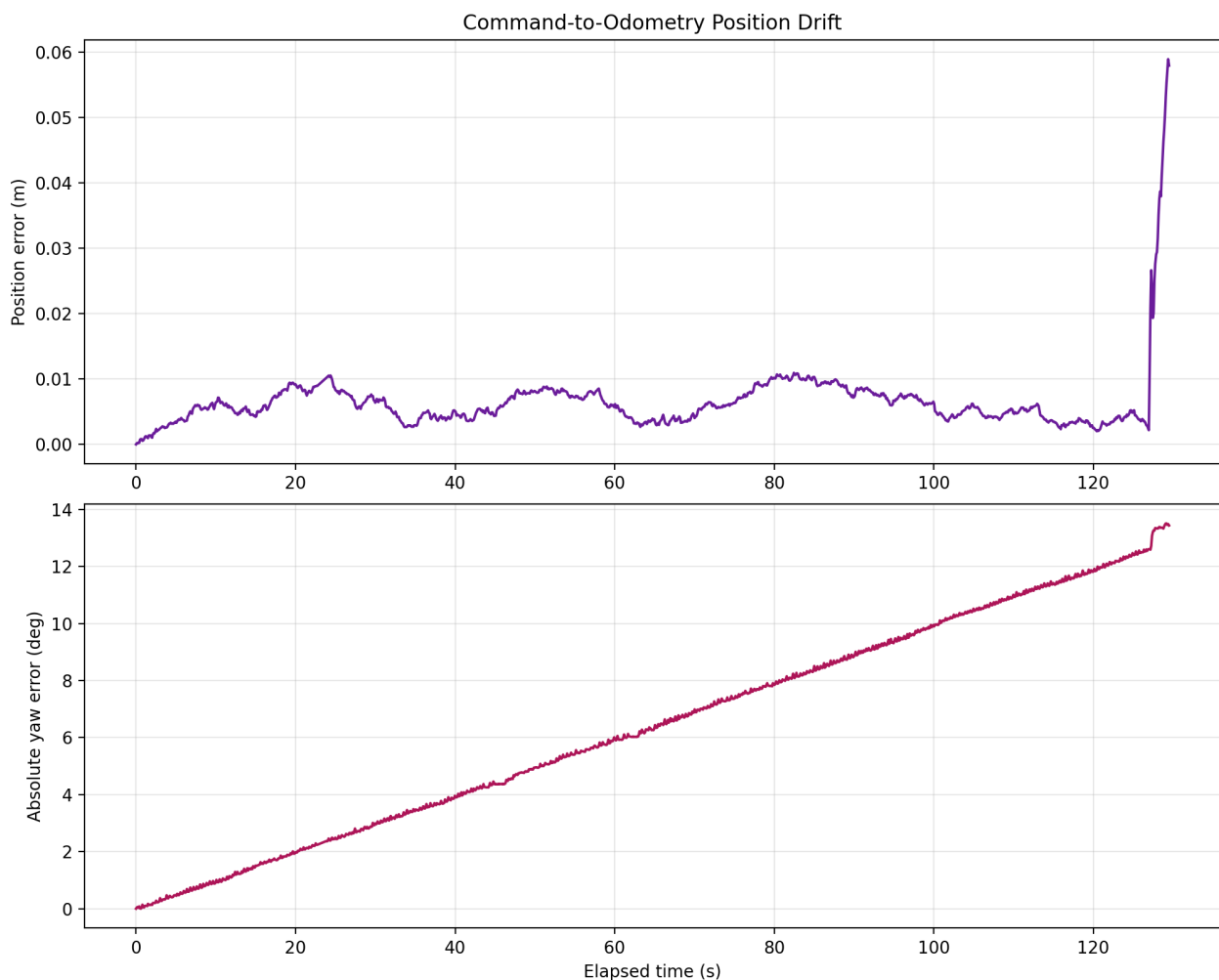


Figure 4. Position and yaw difference between integrated commands and odometry.

Discussion

Position discrepancy stays close to zero for most of the record and rises near the end to approximately 0.058 m. Absolute yaw discrepancy increases gradually and reaches 13.44 degrees. Possible contributors include wheel slip, imperfect wheel-radius or geometry parameters, command transport timing, odometry integration, and the sparse command stream.

Interpretation boundary

This graph is a command-to-odometry consistency check, not an absolute localization accuracy measurement. Only 20 `/cmd_vel` messages were recorded, and the analysis holds each command constant until the next one. A motion-capture system, calibrated fiducial tracking, or another external reference would be required to claim ground-truth accuracy.

Recommended follow-up

- Record `/cmd_vel` at a fixed high rate or include explicit zero commands when keys are released.
- Measure wheel radius and chassis lever arms, then update the driver parameters.
- Repeat straight, lateral, and rotational trials separately to identify direction-specific bias.
- Add an external pose reference and compare it with both wheel odometry and integrated commands.

6. Transform Tree Verification

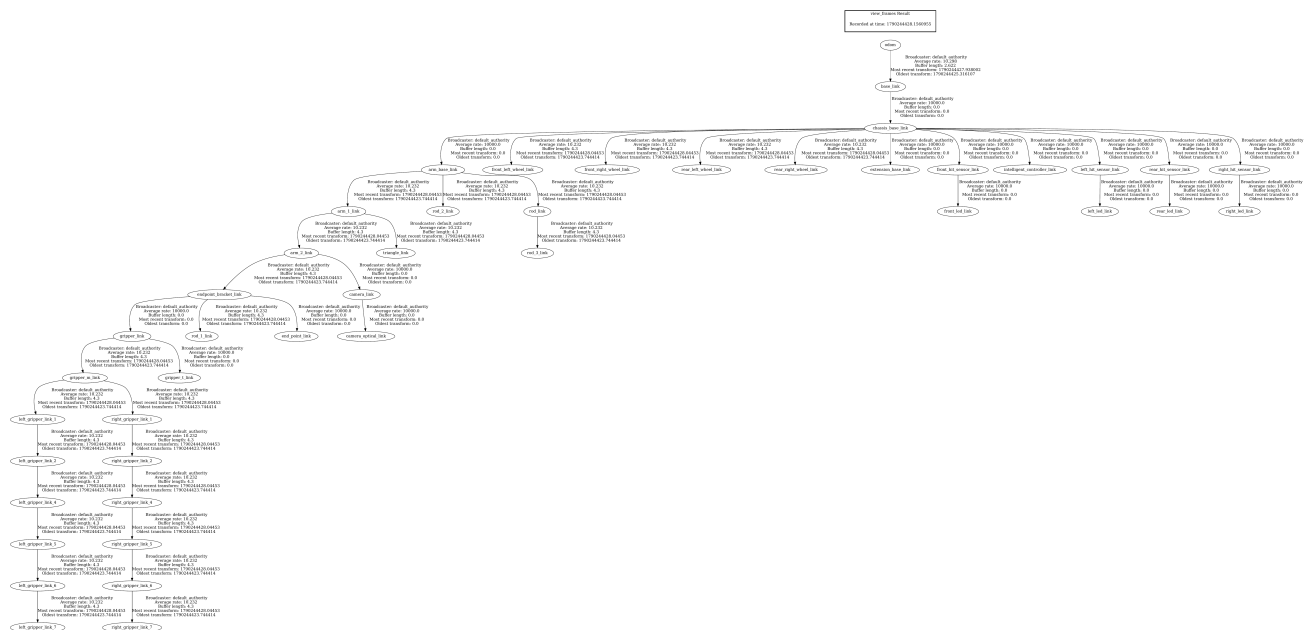


Figure 5. ROS 2 TF tree exported from the live RoboMaster driver.

Frame interpretation

The dynamic transform from odom to base_link represents estimated robot motion. The fixed model then branches through chassis_base_link to wheels, arm, camera, gripper, LEDs, and hit-sensor frames. Wheel and articulated-joint transforms are updated by joint states, while rigid sensor offsets are static.

Relationship	Meaning
odom -> base_link	Time-varying planar robot pose from odometry
base_link -> chassis_base_link	Robot body reference alignment
chassis -> wheel links	Wheel mounting geometry and joint rotation
chassis -> arm/camera/gripper	Manipulator and perception kinematic chain
chassis -> LED/hit-sensor links	Static component mounting locations

Verification result

The tree is connected from odom to every modeled component, with no orphan subtrees. This is the required structural condition for consistent visualization and frame-aware sensor or actuator processing.

7. Mecanum-Wheel Kinematics

7.1 Coordinate convention

Let x point forward, y point left, and positive yaw be counterclockwise. Wheel order is front-left (FL), front-right (FR), rear-left (RL), rear-right (RR). Each wheel has radius r . The wheel center offsets from the chassis origin are l_x longitudinally and l_y laterally; define $L = l_x + l_y$. The sign pattern assumes the standard X-roller configuration. If motor-positive directions differ, the corresponding matrix rows must be negated.

7.2 Inverse kinematics: chassis twist to wheel speeds

$$\begin{bmatrix} w_{FL} \\ w_{FR} \\ w_{RL} \\ w_{RR} \end{bmatrix} = 1/r \begin{bmatrix} 1 & -1 & -L \\ 1 & 1 & L \\ 1 & 1 & -L \\ 1 & -1 & L \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ w_z \end{bmatrix}$$

Forward translation contributes equally to all wheels. Lateral translation reverses the FL and RR signs relative to FR and RL. Rotation produces opposing left/right diagonal contributions proportional to the chassis lever arm L .

7.3 Forward kinematics: wheel speeds to chassis twist

$$\begin{bmatrix} v_x \\ v_y \\ w_z \end{bmatrix} = r/4 \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -1/L & 1/L & -1/L & 1/L \end{bmatrix} \begin{bmatrix} w_{FL} \\ w_{FR} \\ w_{RL} \\ w_{RR} \end{bmatrix}$$

7.4 Why the platform is holonomic

A conventional differential-drive robot cannot instantaneously move sideways; its feasible velocity is constrained to the wheel heading and it is nonholonomic. Angled Mecanum rollers create independently combinable longitudinal, lateral, and rotational velocity components. The four actuators span the three-dimensional planar twist $[v_x, v_y, w_z]$, so the EP can strafe while maintaining its heading.

Desired motion	Wheel-speed sign pattern (FL, FR, RL, RR)
Forward	(+, +, +, +)
Left strafe	(-, +, +, -)
Counterclockwise rotation	(-, +, -, +)

8. Conclusions and Reproduction Guide

8.1 Conclusions

- The standalone SDK controller met the required key mapping, safety stop behavior, and 10 Hz multi-signal logging requirement.
- RNDIS and AP were both reliable in the test, but RNDIS provided far lower and more stable round-trip latency.
- The ROS 2 Jazzy driver built and connected successfully after correcting interpreter selection and installing joint_state_publisher.
- The final MCAP bag contains all required topics and supports quantitative trajectory, drift, and publication-rate analysis.
- The TF tree is connected and reflects the full robot model. The derived Mecanum Jacobian explains the observed holonomic control interface.

8.2 Minimal reproduction sequence

```
# Terminal 1: driver
source ~/rm_venv/bin/activate
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch robomaster_ros main.launch model:=ep conn_type:=ap video:=0 audio:=0

# Terminal 2: record before commanding
cd ~/robomaster_assessment/bags
ros2 bag record -o ros2_ap_run_complete_4 /odom /joint_states /cmd_vel

# Terminal 3: keyboard commands
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# Stop recording with Ctrl-C, then validate and analyze
ros2 bag info ~/robomaster_assessment/bags/ros2_ap_run_complete_4
python scripts/analyze_rosbag.py \
  ~/robomaster_assessment/bags/ros2_ap_run_complete_4 \
  --output-dir ~/robomaster_assessment/results
```

8.3 Submission checklist

Item	Status
Environment and installation documented	Complete
Standalone source and telemetry logs	Complete
100-ping logs for both links	Complete
Network comparison plot and table	Complete
ROS bag with three required topics	Complete
Trajectory and tracking plots	Complete
TF tree and interpretation	Complete
Mecanum derivation	Complete

Appendix A. Key Source Extracts

A.1 Thread-safe telemetry model

```
@dataclass
class Telemetry:
    battery_pct: Optional[float] = None
    yaw_deg: Optional[float] = None
    pitch_deg: Optional[float] = None
    roll_deg: Optional[float] = None
    wheel_rpm: list[Optional[float]] = field(
        default_factory=lambda: [None, None, None, None])
    lock: threading.Lock = field(default_factory=threading.Lock, repr=False)

    def snapshot(self):
        with self.lock:
            return (self.battery_pct, self.yaw_deg, self.pitch_deg,
                    self.roll_deg, *self.wheel_rpm)
```

A.2 Deadline-based 10 Hz logger

```
period = 1.0 / rate_hz
next_sample = time.monotonic()
while not shutdown.is_set():
    now = time.monotonic()
    remaining = next_sample - now
    if remaining > 0:
        shutdown.wait(remaining)
        continue
    writer.writerow([time.time(), now - start_time,
                    *telemetry.snapshot()])
    stream.flush()
    next_sample += period
    if next_sample < now - period:
        next_sample = now + period
```

A.3 Guaranteed shutdown

```
finally:
    shutdown.set()
    try:
        ep_robot.chassis.drive_speed(x=0.0, y=0.0, z=0.0)
        time.sleep(0.2)
    except Exception:
        pass
    # Then quit UI, join logger, unsubscribe, and close robot.
```

References

jeguzzi, robomaster_ros repository and documentation: github.com/jeguzzi/robomaster_ros
Open Robotics, ROS 2 Jazzy documentation: docs.ros.org/en/jazzy
Gazebo, ROS installation guidance for Harmonic: gazebo-sim.org/docs/harmonic/ros_installation
DJI RoboMaster Python SDK documentation: robomaster-dev.readthedocs.io
NumPy troubleshooting guide, C-API incompatibility section: numpy.org/doc/stable/user/troubleshooting-importerror.html
ros-teleop, teleop_twist_keyboard documentation: github.com/ros-teleop/teleop_twist_keyboard